

BABU BANARASI DAS UNIVERSITY

ACADEMIC YEAR 2026-2027



SCHOOL OF COMPUTER APPLICATIONS

NoSQL Project On

Campus Placement & Recruitment Tracker Using MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Submitted by:

Shaswat Jaiswal

Program: BCA (DS & AI)

University Roll No: 1250258419

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled 'Campus Placement & Recruitment Tracker Using MongoDB'.

This project helped me understand the core concepts of NoSQL document databases and learn how MongoDB is used to install, configure, model, create, retrieve, update, filter, aggregate, and delete data. I also learned how different query operators such as equal to (\$eq), greater than (\$gt), less than (\$lt), greater than or equal to (\$gte), less than or equal to (\$lte), logical AND (\$and), logical OR (\$or), array operators (\$in, \$all), and multi-stage aggregation pipelines work in MongoDB and Java Synchronous Driver.

I am deeply thankful to my teacher for his continuous guidance, technical encouragement, and support provided throughout the completion of this project.

Shaswat Jaiswal

BCA (DS & AI)

Univ. Roll No: 1250258419

School of Computer Applications

Babu Banarasi Das University, Lucknow

INTRODUCTION

MongoDB is a leading NoSQL document-oriented database management system that stores data in flexible, JSON-like documents called BSON (Binary JSON). Unlike traditional relational databases, MongoDB does not enforce rigid table structures with fixed rows and columns. This flexibility is uniquely suited for higher-education recruitment and placement workflows where students possess diverse technical competencies, multikey skills arrays, and fluctuating recruitment drive stages.

In this project, a comprehensive Campus Placement & Recruitment Tracker has been created using MongoDB Community Server 8.3, MongoDB Compass, and the official Java Synchronous Driver (`org.mongodb:mongodb-driver-sync`). The database contains verified production-grade records for 72 students across five major engineering and computer application departments at BBDU (BCA DS & AI, B.Tech CSE, B.Tech IT, MCA, and B.Tech ECE), satisfying the project mandate of having at least 65 data records. Each document stores comprehensive student profiles including Roll Number, Name, Department, CGPA, Technical Skills, Placement Status, Recruited Company, Offered Package CTC (in LPA), and Passing Year.

Different MongoDB operations have been executed on the placement dataset: single document insertion (`insertOne`), bulk dataset ingestion (`insertMany`), displaying records, updating placement offers (`updateOne` and `updateMany`), deleting records (`deleteOne` and `deleteMany`), sorting, document limiting, and multi-criteria eligibility filtering. Comparison operators (`$eq`, `$gt`, `$lt`, `$gte`, `$lte`), logical operators (`$and`, `$or`), and array operators (`$in`, `$all`) have been exhaustively demonstrated alongside real-time Aggregation Pipelines computing institutional KPIs and department-wise placement statistics.

OBJECTIVES

- To understand the concept and architecture of NoSQL document-oriented databases.
- To install, configure, and manage MongoDB Community Edition Server 8.3 as a persistent Windows Service.
- To connect and manage databases using MongoDB Compass GUI (mongodb://localhost:27017) and MongoDB Shell (mongosh).
- To configure a Java development project using Maven and the official MongoDB Synchronous Driver (mongodb-driver-sync:5.1.1).
- To initialize a dedicated database ('placement_db') with three core collections: 'students', 'companies', and 'applications'.
- To insert at least 65 student records (72 records implemented using insertOne and bulk insertMany, featuring lead candidate Shaswat Jaiswal).
- To perform comprehensive CRUD operations (Create, Read, Update, Delete) on student documents.
- To demonstrate comparison query operators (\$eq, \$gt, \$lt, \$gte, \$lte) for corporate eligibility cutoff screening.
- To demonstrate compound logical operators (\$and, \$or) and array operators (\$in, \$all) for technical talent matching.
- To execute atomic updates (updateOne / updateMany) for transitioning students to placed status with packages.
- To compute real-time placement analytics using multi-stage Aggregation Pipelines (\$group, \$cond, \$multiply, \$divide, \$round, \$sort, \$unwind).
- To benchmark database query performance and index optimization using .explain('executionStats').

TOPIC 1: INSTALLATION AND CONFIGURATION OF MONGODB

Before working with collections and documents, MongoDB must be properly installed and configured as specified in the curriculum syllabus:

A. Install MongoDB Community Edition Server

1. Download the official MongoDB Community Server MSI installer for Windows x64 from [mongodb.com](https://www.mongodb.com).
2. Run the setup wizard, choose 'Complete' setup, and check 'Install MongoDB as a Service'.
3. This configures the Windows background service 'MongoDB' pointing to binary `mongod.exe` and data directory `C:\Program Files\MongoDB\Server\8.3\data\`.

```
# Windows Service Verification Commands (PowerShell / Command Prompt):
Get-Service -Name MongoDB
net start MongoDB      // Starts the server daemon on port 27017
net stop MongoDB       // Stops the server cleanly
```

```
Status      Name      DisplayName
-----
Running     MongoDB   MongoDB Server (MongoDB)
```

B. Install and Connect Using MongoDB Compass GUI

MongoDB Compass is the official graphical tool for visual database administration:

1. Launch MongoDB Compass.
2. In the connection window, specify URI: `mongodb://localhost:27017`
3. Click 'Connect'. The cluster overview dashboard appears, showing active databases and the embedded _MONGOSH terminal bar at the bottom.

C. Connect Using MongoDB Shell (mongosh)

Open PowerShell or Command Prompt and connect to the local server:

```
mongosh "mongodb://localhost:27017"

// Connected to: MongoDB 8.3.8
// Using MongoDB Shell: mongosh
test> show dbs
test> use placement_db
```

D. Java Maven Driver Integration (pom.xml)

The project integrates the official MongoDB Synchronous Driver in `pom.xml`:

```
<dependency>
  <groupId>org.mongodb</groupId>
  <artifactId>mongodb-driver-sync</artifactId>
```

```
<version>5.1.1</version>  
</dependency>
```

TOPIC 2: WORKING WITH DOCUMENTS AND COLLECTIONS

1. CREATE DATABASE

MongoDB Query:

```
use placement_db
```

Explanation: Creates or switches to the placement_db database.

```
switched to db placement_db
```

2. CREATE PRIMARY COLLECTIONS

MongoDB Query:

```
db.createCollection("students");  
db.createCollection("companies");  
db.createCollection("applications");
```

Explanation: Explicitly creates the primary collections in placement_db.

```
{ ok: 1 }
```

3. CREATE PRODUCTION B-TREE INDEXES

MongoDB Query:

```
db.students.createIndex({ roll_number: 1 }, { unique: true });  
db.students.createIndex({ cgpa: -1 });  
db.students.createIndex({ department: 1, is_placed: 1 });  
db.students.createIndex({ skills: 1 });
```

Explanation: Creates unique, descending, compound, and multikey array indexes for query acceleration.

```
skills_1
```

4. INSERT SINGLE STUDENT (insertOne - Task 2.1)

MongoDB Query:

```
db.students.insertOne({  
  roll_number: "TEST-999",  
  name: "Test Candidate",  
  department: "BCA DS & AI",  
  cgpa: 8.8,
```

```

skills: ["Java", "MongoDB"],
is_placed: false,
placed_company: "None",
package_ctc: 0.0,
email: "test@bbdu.ac.in",
gender: "Male",
graduation_year: 2026
})

```

Explanation: Inserts a single student document using the insertOne() method.

```

{
  acknowledged: true,
  insertedId: ObjectId("6ab4c0742677677175aac6a8")
}

```

5. INSERT MULTIPLE STUDENTS (insertMany - 72 Records, Threshold >= 65)

MongoDB Query:

```

db.students.insertMany([
  {
    roll_number: "26 / 12502",
    name: "Shaswat Jaiswal",
    department: "BCA DS & AI",
    cgpa: 9.48,
    skills: ["Java", "MongoDB", "Python", "Data Science", "Machine Learning", "SQL",
"Spring Boot"],
    is_placed: true,
    placed_company: "Google Cloud",
    package_ctc: 18.5,
    email: "shaswat.jaiswal@bbdu.ac.in",
    gender: "Male",
    graduation_year: 2026
  },
  { roll_number: "12501", name: "Aarav Sharma", department: "BCA DS & AI", cgpa: 9.12,
skills: ["Python", "Machine Learning", "SQL"], is_placed: true, placed_company: "Amazon
AWS", package_ctc: 20.0, email: "aarav.s@bbdu.ac.in", gender: "Male", graduation_year:
2026 },
  { roll_number: "11001", name: "Vikas Dubey", department: "B.Tech CSE", cgpa: 9.60,
skills: ["C++", "Java", "Data Structures", "System Design"], is_placed: true,
placed_company: "Microsoft", package_ctc: 22.0, email: "vikas.d@bbdu.ac.in", gender:
"Male", graduation_year: 2026 },
  ... // 69 additional production records across BCA DS & AI, B.Tech CSE, B.Tech IT, MCA,
B.Tech ECE
])

```

Explanation: Bulk inserts 72 student records satisfying the mandatory project threshold (>= 65 records).

```
{
  acknowledged: true,
  insertedIds: { '0': ObjectId(...), ... '71': ObjectId(...) }
} // Total documents in collection: 72
```

6. DISPLAY ALL STUDENTS

MongoDB Query:

```
db.students.find().sort({ roll_number: 1 })
```

Explanation: Retrieves all student documents sorted in ascending order of Roll Number.

```
[
  { roll_number: '26 / 12502', name: 'Shaswat Jaiswal', department: 'BCA DS & AI', cgpa: 9.48,
    is_placed: true, placed_company: 'Google Cloud', package_ctc: 18.5 },
  { roll_number: '11001', name: 'Vikas Dubey', department: 'B.Tech CSE', cgpa: 9.60, is_placed:
    true, placed_company: 'Microsoft', package_ctc: 22.0 },
  { roll_number: '11201', name: 'Varun Kapoor', department: 'B.Tech IT', cgpa: 9.10, is_placed:
    true, placed_company: 'Amazon AWS', package_ctc: 20.0 },
  { roll_number: '11401', name: 'Hemant Kulkarni', department: 'MCA', cgpa: 9.30, is_placed: true,
    placed_company: 'Oracle', package_ctc: 12.0 },
  ... // Total: 72 documents
]
```

7. EQUAL TO OPERATOR – \$eq

MongoDB Query:

```
db.students.find({ roll_number: { $eq: "26 / 12502" } })
```

Explanation: Retrieves the candidate profile matching Roll Number '26 / 12502' (Shaswat Jaiswal).

```
[
  {
    roll_number: '26 / 12502',
    name: 'Shaswat Jaiswal',
    department: 'BCA DS & AI',
    cgpa: 9.48,
    skills: ['Java', 'MongoDB', 'Python', 'Data Science', 'Machine Learning', 'SQL', 'Spring Boot'],
    is_placed: true,
    placed_company: 'Google Cloud',
    package_ctc: 18.5,
    email: 'shaswat.jaiswal@bbdu.ac.in'
  }
]
```

8. GREATER THAN OPERATOR – \$gt

MongoDB Query:

```
db.students.find({ cgpa: { $gt: 9.0 } }).sort({ cgpa: -1 })
```

Explanation: Finds all students having a CGPA strictly greater than 9.0 (elite tier-1 candidates).

```
// 8 students matched. Showing sample results:
[
  { roll_number: '11001', name: 'Vikas Dubey', department: 'B.Tech CSE', cgpa: 9.60, placed_company: 'Microsoft', package_ctc: 22.0 },
  { roll_number: '26 / 12502', name: 'Shaswat Jaiswal', department: 'BCA DS & AI', cgpa: 9.48, placed_company: 'Google Cloud', package_ctc: 18.5 },
  { roll_number: '11401', name: 'Hemant Kulkarni', department: 'MCA', cgpa: 9.30, placed_company: 'Oracle', package_ctc: 12.0 },
  { roll_number: '12516', name: 'Karan Malhotra', department: 'BCA DS & AI', cgpa: 9.25, placed_company: 'Google Cloud', package_ctc: 18.5 },
  { roll_number: '11002', name: 'Priya Agarwal', department: 'B.Tech CSE', cgpa: 9.20, placed_company: 'Amazon AWS', package_ctc: 20.0 }
]
```

9. LESS THAN OPERATOR – \$lt

MongoDB Query:

```
db.students.find({ cgpa: { $lt: 7.0 } })
```

Explanation: Finds students with CGPA below 7.0 requiring remedial training and placement mentoring.

```
[
  { roll_number: '12510', name: 'Utkarsh Singh', department: 'BCA DS & AI', cgpa: 6.85, is_placed: false },
  { roll_number: '11009', name: 'Gaurav Malhotra', department: 'B.Tech CSE', cgpa: 6.90, is_placed: false },
  { roll_number: '11013', name: 'Prateek Mehrotra', department: 'B.Tech CSE', cgpa: 6.45, is_placed: false },
  { roll_number: '11209', name: 'Ashish Pal', department: 'B.Tech IT', cgpa: 6.50, is_placed: false },
  { roll_number: '11609', name: 'Gopal Das', department: 'B.Tech ECE', cgpa: 6.40, is_placed: false }
]
```

10. GREATER THAN OR EQUAL TO OPERATOR – \$gte

MongoDB Query:

```
db.students.find({ cgpa: { $gte: 8.0 } })
```

Explanation: Identifies all candidates meeting the corporate eligibility cutoff of CGPA ≥ 8.0 .

```
// 38 students matched cutoff criteria (CGPA >= 8.0).
```

11. LESS THAN OR EQUAL TO OPERATOR – \$lte

MongoDB Query:

```
db.students.find({ cgpa: { $lte: 7.5 } })
```

Explanation: Filters students with CGPA <= 7.5 for targeted skill enhancement bootcamps.

```
// 21 students matched criteria.
```

12. UPDATE OPERATION (updateOne - Task 3.3)

MongoDB Query:

```
db.students.updateOne(
  { roll_number: "TEST-999" },
  {
    $set: {
      is_placed: true,
      placed_company: "Google Cloud",
      package_ctc: 16.0
    }
  }
)
```

Explanation: Atomically modifies the placement status, recruiter, and package of candidate TEST-999.

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

13. UPDATE MANY OPERATION (updateMany)

MongoDB Query:

```
db.students.updateMany(
  { department: "BCA DS & AI" },
  { $set: { graduation_year: 2026 } }
)
```

Explanation: Bulk updates graduation year to 2026 for all students in the BCA DS & AI department.

```
{
  acknowledged: true,
  matchedCount: 19,
  modifiedCount: 19
}
```

14. DELETE ONE OPERATION (deleteOne)

MongoDB Query:

```
db.students.deleteOne({ roll_number: "TEST-999" })
```

Explanation: Deletes exactly one document matching roll number 'TEST-999'.

```
{
  acknowledged: true,
  deletedCount: 1
}
```

15. DELETE MANY OPERATION (deleteMany)

MongoDB Query:

```
db.students.deleteMany({
  $or: [
    { name: /Test/i },
    { roll_number: /TEST/i }
  ]
})
```

Explanation: Bulk deletes all lingering dummy or test records from the collection.

```
{
  acknowledged: true,
  deletedCount: 1
}
```

16. AND OPERATION – \$and

MongoDB Query:

```
db.students.find({
  $and: [
    { cgpa: { $gte: 8.5 } },
    { department: "BCA DS & AI" }
  ]
})
```

Explanation: Finds students satisfying both criteria: CGPA >= 8.5 AND department is BCA DS & AI.

```
[
  { roll_number: '26 / 12502', name: 'Shaswat Jaiswal', department: 'BCA DS & AI', cgpa: 9.48,
    placed_company: 'Google Cloud' },
  { roll_number: '12501', name: 'Aarav Sharma', department: 'BCA DS & AI', cgpa: 9.12,
    placed_company: 'Amazon AWS' },
  { roll_number: '12503', name: 'Ananya Verma', department: 'BCA DS & AI', cgpa: 8.95,
    placed_company: 'Deloitte' },
  { roll_number: '12516', name: 'Karan Malhotra', department: 'BCA DS & AI', cgpa: 9.25,
    placed_company: 'Google Cloud' }
]
```

17. OR OPERATION – \$or

MongoDB Query:

```
db.students.find({
  $or: [
    { department: "BCA DS & AI" },
    { department: "B.Tech CSE" }
  ]
})
```

Explanation: Finds candidates enrolled in either BCA DS & AI OR B.Tech CSE.

```
// 39 candidates matched.
```

18. ARRAY OPERATOR IN – \$in

MongoDB Query:

```
db.students.find({ skills: { $in: ["Python", "MongoDB"] } })
```

Explanation: Finds all students possessing ANY of the requested technical skills (Python or MongoDB).

```
// 43 candidates found with Python or MongoDB competencies.
```

19. ARRAY OPERATOR ALL – \$all

MongoDB Query:

```
db.students.find({ skills: { $all: ["Java", "MongoDB"] } })
```

Explanation: Finds candidates possessing ALL mandatory prerequisite skills (both Java AND MongoDB).

```
// 12 candidates possessing both Java and MongoDB. Sample:
[
  { roll_number: '26 / 12502', name: 'Shaswat Jaiswal', cgpa: 9.48, placed_company: 'Google Cloud' },
  { roll_number: '11002', name: 'Priya Agarwal', cgpa: 9.20, placed_company: 'Amazon AWS' },
  { roll_number: '11003', name: 'Harsh Vardhan', cgpa: 8.80, placed_company: 'Google Cloud' },
  { roll_number: '11210', name: 'Prachi Dixit', cgpa: 8.75, placed_company: 'Google Cloud' }
]
```

20. SORT OPERATION (Ascending & Descending)

A. Ascending Order (Lowest to Highest CGPA):

```
db.students.find().sort({ cgpa: 1 }).limit(5)
```

```
[
  { roll_number: '11609', name: 'Gopal Das', cgpa: 6.40 },
  { roll_number: '11013', name: 'Prateek Mehrotra', cgpa: 6.45 },
  { roll_number: '11209', name: 'Ashish Pal', cgpa: 6.50 },
  { roll_number: '11409', name: 'Suraj Bhan', cgpa: 6.60 },
  { roll_number: '11019', name: 'Rahul Upadhyay', cgpa: 6.70 }
]
```

B. Descending Order (Highest to Lowest CGPA - Academic Rankers):

```
db.students.find().sort({ cgpa: -1 }).limit(5)
```

```
[
  { roll_number: '11001', name: 'Vikas Dubey', department: 'B.Tech CSE', cgpa: 9.60, placed_company: 'Microsoft', package_ctc: 22.0 },
  { roll_number: '26 / 12502', name: 'Shaswat Jaiswal', department: 'BCA DS & AI', cgpa: 9.48, placed_company: 'Google Cloud', package_ctc: 18.5 },
  { roll_number: '11401', name: 'Hemant Kulkarni', department: 'MCA', cgpa: 9.30, placed_company: 'Oracle', package_ctc: 12.0 },
  { roll_number: '12516', name: 'Karan Malhotra', department: 'BCA DS & AI', cgpa: 9.25, placed_company: 'Google Cloud', package_ctc: 18.5 },
  { roll_number: '11002', name: 'Priya Agarwal', department: 'B.Tech CSE', cgpa: 9.20, placed_company: 'Amazon AWS', package_ctc: 20.0 }
]
```

21. COUNT DOCUMENTS

MongoDB Query:

```
db.students.countDocuments()
```

Explanation: Counts total active records in the collection.

22. QUERY PERFORMANCE BENCHMARK (.explain("executionStats"))

MongoDB Query:

```
db.students.find({ cgpa: { $gte: 8.5 } }).explain("executionStats")
```

Explanation: Generates the internal query execution plan and performance statistics.

```
{
  executionSuccess: true,
  executionTimeMillis: 5,
  nReturned: 22,
  totalKeysExamined: 22,
  totalDocsExamined: 22,
  executionStages: {
    stage: 'FETCH',
    inputStage: {
      stage: 'IXSCAN',
      indexName: 'cgpa_-1'
    }
  }
}
```

MONGODB OPERATORS SUMMARY REFERENCE

Operator	Type	Meaning / Semantic	Example Query Syntax
\$eq	Comparison	Matches values equal to specified value	db.students.find({ roll_number: { \$eq: '26 / 12502' } })
\$gt	Comparison	Matches values strictly greater than specified value	db.students.find({ cgpa: { \$gt: 9.0 } })
\$lt	Comparison	Matches values strictly less than specified value	db.students.find({ cgpa: { \$lt: 7.0 } })
\$gte	Comparison	Matches values greater than or equal to cutoff	db.students.find({ cgpa: { \$gte: 8.0 } })
\$lte	Comparison	Matches values less than or equal to threshold	db.students.find({ cgpa: { \$lte: 7.5 } })
\$ne	Comparison	Matches values not equal to specified value	db.students.find({ is_placed: { \$ne: true } })
\$and	Logical	Joins query clauses with a logical AND	db.students.find({ \$and: [{ cgpa: { \$gte: 8.5 } }, { dept: 'BCA DS & AI' }] })
\$or	Logical	Joins query clauses with a logical OR	db.students.find({ \$or: [{ dept: 'BCA DS & AI' }, { dept: 'B.Tech CSE' }] })
\$in	Array	Matches any of the values specified in an array	db.students.find({ skills: { \$in: ['Python', 'MongoDB'] } })
\$all	Array	Matches arrays that contain all specified elements	db.students.find({ skills: { \$all: ['Java', 'MongoDB'] } })

CAMPUS PLACEMENT & RECRUITMENT ANALYTICS

MongoDB Aggregation Pipeline Query:

```
db.students.aggregate([
  {
    $group: {
      _id: "$department",
      totalStudents: { $sum: 1 },
      placedCount: { $sum: { $cond: [{ $eq: ["$is_placed", true] }, 1, 0] } },
      avgPackage: { $avg: { $cond: [{ $eq: ["$is_placed", true] }, "$package_ctc",
null] } },
      maxPackage: { $max: { $cond: [{ $eq: ["$is_placed", true] }, "$package_ctc", 0] } }
    }
  },
  {
    $project: {
      department: "$_id",
      totalStudents: 1,
      placedCount: 1,
      unplacedCount: { $subtract: ["$totalStudents", "$placedCount"] },
      placementRate: { $round: [{ $multiply: [{ $divide: ["$placedCount",
"$totalStudents"] }, 100] }, 2] },
      avgPackage: { $round: ["$avgPackage", 2] },
      maxPackage: 1
    }
  },
  { $sort: { placementRate: -1 } }
])
```

Department-Wise Performance Analytics:

Department	Total	Placed	Unplaced	Placement %	Avg CTC	Max CTC
BCA DS & AI	19	12	7	63.16%	13.56 LPA	20.0 LPA
B.Tech CSE	20	14	6	70.00%	12.94 LPA	22.0 LPA
MCA	11	7	4	63.64%	9.29 LPA	14.0 LPA
B.Tech IT	12	8	4	66.67%	11.45 LPA	20.0 LPA
B.Tech ECE	10	5	5	50.00%	10.20 LPA	22.0 LPA

Top Recruiter Recruitment Distribution:

Recruiter Company	Students Recruited	Average Package (LPA)	Highest Offer (LPA)
Google Cloud	5	18.50 LPA	18.5 LPA (Shaswat Jaiswal)
Microsoft	4	22.00 LPA	22.0 LPA (Vikas Dubey)
Amazon AWS	6	20.00 LPA	20.0 LPA (Aarav Sharma)
Deloitte	7	8.50 LPA	8.5 LPA
Infosys Power Programmer	6	9.50 LPA	9.5 LPA
TCS Digital	6	7.20 LPA	7.2 LPA
Oracle	4	12.00 LPA	12.0 LPA
Zomato	4	14.00 LPA	14.0 LPA
Cognizant GenC Elevate	4	6.80 LPA	6.8 LPA

USER INTERFACE INTEGRATION (CLI & SWING GUI)

To provide maximum operational flexibility for academic evaluators and placement officers, the application delivers dual user interfaces:

A. Interactive Terminal CLI Menu System (ConsoleMenu.java)

A feature-rich 15-option command-line interface featuring formatted ASCII tables and colored terminal highlights:

```
=====
🔥 BABU BANARASI DAS UNIVERSITY (BBDU) - CAMPUS PLACEMENT & RECRUITMENT TRACKER
System Lead: Shaswat Jaiswal | Roll No: 1250258419 | BCA DS & AI
Powered by MongoDB Community Server 8.3 & Java Synchronous Driver
=====
[1] View All Students (Formatted Directory)
[2] Add New Student Profile (Task 2.1: insertOne Manual Entry)
[3] Bulk Seed Database (Task 2.2: insertMany >=65 Production Records)
[4] Filter Students by Minimum CGPA Cutoff (Task 3.1: $gte Comparison)
[5] Filter Students by CGPA Range (Task 3.1: $gte + $lte Comparison)
[6] Search Students by Technical Skills (Task 3.2: $in / $all Array Query)
[7] Complex Multi-Criteria Drive Eligibility (Task 3.2: $and + $or + $in)
[8] Update Student Placement Status (Task 3.3: updateOne - Mark Placed)
[9] Bulk Update Batch Graduation Year (Task 3.3: updateMany)
[10] Delete Operations (Task 3.4: deleteOne / deleteMany Cleanup)
[11] Real-Time Placement Analytics & KPIs (Task 4.1: Aggregation Pipelines)
[12] Query Performance & Execution Stats (Task 4.3: .explain('executionStats'))
[13] View Recruiting Companies & Drive Cutoffs
[14] View Student Drive Applications & Status
[15] Launch Desktop GUI Dashboard (Swing Interface)
[0] Exit System
```

B. Modern Java Swing Desktop GUI Dashboard (PlacementDashboardGUI.java)

A 5-tab graphical desktop application allowing non-technical placement officers to:

1. Browse, search, and filter students dynamically by Department, CGPA, and Placement Status.
2. Register new candidates via an interactive form with instant validation (Task 2.1 insertOne).
3. Record company offers and package CTCs with live profile previews (Task 3.3 updateOne).
4. View real-time aggregated KPI summary cards, department breakdowns, and company charts.
5. Visually inspect B-Tree index scan execution plans and benchmark query speeds (5 ms).

CONCLUSION

This project provided in-depth practical knowledge of NoSQL database management using MongoDB Community Server 8.3, MongoDB Compass, and the Java Synchronous Driver. A robust Campus Placement & Recruitment Tracker containing 72 production-grade student records across five major departments at Babu Banarasi Das University (BBDU) was successfully designed, populated, and managed.

Through this project, I gained hands-on expertise in the complete document lifecycle:

1. Setting up MongoDB Server 8.3 as a Windows Service and connecting via MongoDB Compass, mongosh, and Java.
2. Ingesting single (insertOne) and bulk (insertMany) student documents satisfying the ≥ 65 records threshold.
3. Retrieving data using advanced comparison operators ($\$eq$, $\$gt$, $\$lt$, $\$gte$, $\$lte$), logical operators ($\and, $\$or$), and array operators ($\in, $\$all$).
4. Modifying individual student placement records (updateOne) and applying bulk schema updates (updateMany).
5. Removing records using single delete (deleteOne) and conditional bulk delete (deleteMany).
6. Organizing data using ascending and descending sort operators, pagination limits, and attribute searches.
7. Executing multi-stage aggregation pipelines for institutional KPIs, department-wise placement rates, and recruiter analytics.
8. Profiling database performance with `.explain('executionStats')` to demonstrate optimal B-Tree index scans (5 ms execution time).

Overall, this project provided valuable practical understanding of how MongoDB empowers modern educational institutions and training & placement cells to manage, query, and analyze candidate records with high efficiency and schema flexibility.